MODULE 28 · DOMAIN 4 - TROUBLESHOOTING AND OPTIMIZATION

Performance and Cost Optimisation

Why more memory is often cheaper, where a serverless bill actually goes, and the line item nobody predicted.

DOMAIN 4 of 4 EXAM WEIGHT 18% MODULE 28 of 34

admin@sthithapragna.com

On servers, speed costs money. In serverless, you pay for **time**, so faster is usually **cheaper**.

Memory and CPU in Lambda

- **Memory is the only performance setting.** CPU scales proportionally with it.
- At 128 MB a function gets a fraction of a vCPU. At **1,769 MB** it gets a full one.
- Network bandwidth scales with memory too, so uploads and downloads speed up.
- **You pay per GB-second**, so doubling memory doubles the rate and may halve the duration.
- **The cheapest setting is often not the lowest.** That is the counterintuitive part.

> READ THIS TWICE
>
> The arithmetic: 128 MB for 800 ms is 0.1 GB-seconds. 512 MB for 150 ms is 0.075 GB-seconds. Four times the memory, and it costs less.

Lambda Power Tuning

- A Step Functions state machine that runs your function at several memory settings.
- It plots duration and cost against memory, so the optimum is visible rather than guessed.
- Strategies: optimise for **cost**, for **speed**, or for a **balanced** weighting.
- The optimum is almost never where a team first set it, which is usually 128 MB by default.
- Re-run it after significant code changes. The curve moves.

> ON THE EXAM
>
> It is a Step Functions application you deploy from the Serverless Application Repository, which makes it the answer when a scenario asks how to determine the right memory setting.

Graviton

- Setting the architecture to **arm64** runs the function on AWS Graviton2.
- Duration charges are **20% lower**, and performance is up to 19% better.
- AWS quotes up to **34% better price performance** from the two combined.
- Most runtimes need no code change at all. Interpreted languages are the easiest.
- Native binaries and architecture-specific dependencies must be rebuilt for ARM.

> ON THE EXAM
>
> This is the rare change that costs nothing to make and reduces the bill immediately. The only real risk is a compiled dependency that has no ARM build.

Where a serverless bill actually goes

- **Lambda compute is often only 20 to 40% of a serverless bill.**
- **CloudWatch Logs** ingestion and retention are frequently larger than the compute.
- **NAT gateway** charges appear for any VPC-attached function reaching AWS services.
- **API Gateway** at 3.50 dollars per million requests can exceed the Lambda cost behind it.
- Data transfer sits under **EC2-Other** in Cost Explorer, which hides it from casual reading.

ON THE EXAM

Optimising Lambda memory on a bill where logs cost more than compute is effort in the wrong place. Look at the breakdown before choosing a lever.

Batching and payload size

- **BatchWriteItem** and **BatchGetItem** reduce DynamoDB request overhead.
- **SQS batch send and receive** handle up to 10 messages per API call.
- SQS bills per request in **64 KB chunks**, so a large message is several billable requests.
- A Lambda event source mapping with a larger **batch size** means fewer invocations.
- Larger batches raise the cost of a failure, since a whole batch retries together.

> ON THE EXAM
>
> The trade-off is real: bigger batches are cheaper per item and more expensive per failure, which is why partial batch responses matter.

Cold starts, revisited as cost

- Initialisation time is **billed** since August 2025, so a slow init costs money on every cold start.
- Smaller deployment packages initialise faster. Trim dependencies rather than adding memory.
- Instantiate SDK clients **outside the handler**, so they are reused across invocations.
- **SnapStart** removes most init time for supported runtimes, at no extra charge.
- **Provisioned concurrency** removes cold starts by paying for idle capacity.

> ON THE EXAM
>
> The ordering matters: trim the package, move client creation out of the handler, then consider SnapStart, and only then pay for provisioned concurrency.

The provisioned concurrency trade

- You pay for provisioned environments **whether or not they are used**.
- At steady high utilisation the per-invocation rate is lower, so it can be cheaper.
- At low or spiky utilisation you are paying for idle capacity, so it is dearer.
- **Scheduled scaling** provisions only during known busy hours, which fits predictable traffic.
- Buying it to fix occasional cold starts on a quiet function is the classic overspend.

> ON THE EXAM
>
> The question to ask is whether the traffic is predictable. Provisioned concurrency rewards predictability and punishes everything else.

DynamoDB cost levers

- **On-demand** for unpredictable traffic. **Provisioned with auto scaling** for steady traffic.
- Provisioned is substantially cheaper at consistent utilisation, and **reserved capacity** cheaper still.
- **Projections** matter: a KEYS_ONLY index costs far less to maintain than ALL.
- **TTL** deletes expired items automatically, and the deletions are free.
- Eventually consistent reads cost **half** what strongly consistent reads cost.

> ON THE EXAM
>
> TTL is the underused one. It removes both storage cost and the read cost of scanning past expired items, and it charges nothing for the deletions.

S3 cost levers

- **Lifecycle rules** transition to cheaper classes and expire objects on a schedule.
- **Intelligent-Tiering** moves objects automatically based on access, with no retrieval fees.
- **AbortIncompleteMultipartUpload** removes storage you are billed for and cannot see.
- **S3 Bucket Keys** cut SSE-KMS request costs by up to 99%.
- Expiring **noncurrent versions** is essential on any versioned bucket.

> ON THE EXAM
>
> Intelligent-Tiering is the answer when access patterns are unknown or changing. Lifecycle rules are better when you already know the pattern.

CloudWatch cost levers

- **Ingestion is around 0.50 dollars per GB**, and storage around 0.03 dollars per GB per month.
- **Log groups never expire by default**, so every log line ever written is still billed.
- Lambda's **application log level** filters before ingestion, so filtered lines cost nothing.
- Set a retention period on **every** log group, and export to S3 for long-term archival.
- High-cardinality **custom metric dimensions** multiply into thousands of billable metrics.

> ON THE EXAM
>
> Debug logging left on in production is the most common single line item people are surprised by, and filtering at the platform is free to configure.

The front door and the plumbing

- **HTTP APIs cost around 71% less** than REST APIs. Module 11 listed what you give up.
- An **Application Load Balancer** is cheaper still at high volume, with fewer features.
- **API Gateway caching** is billed hourly whether or not it gets hits.
- **EventBridge** costs more per million events than **SQS**. Direct S3 triggers are free.
- SQS long polling removes the empty receives you would otherwise pay for.

> ON THE EXAM
>
> The front door is often overlooked because it feels like plumbing. At high request volume it can cost more than the compute it fronts.

Data transfer

- Inbound data transfer is free. **Outbound to the internet is charged**, by the gigabyte.
- **Cross-Availability-Zone traffic is charged in both directions**, even inside one VPC.
- **Cross-Region traffic costs more.** Replication is a recurring charge, not a one-off.
- **CloudFront** reduces internet egress cost as well as latency.
- Keeping chatty components in the same Availability Zone removes a real line item.

> ON THE EXAM
>
> The cross-AZ charge surprises people because it is invisible in the architecture diagram. Two services talking constantly across zones pay for every byte, twice.

NAT gateways and VPC endpoints

- A NAT gateway costs roughly **32 dollars a month per Availability Zone**, plus per-GB processing.
- A VPC-attached function reaching S3 or DynamoDB routes through it by default.
- **Gateway endpoints** for S3 and DynamoDB are **free**, with no per-GB charge.
- **Interface endpoints** for other services cost roughly 7 dollars a month plus a small per-GB fee.
- If a function only calls AWS services, the NAT gateway can be removed entirely.

> ON THE EXAM
>
> The gateway endpoint being free is the detail to remember. A VPC-attached function talking to S3 through a NAT gateway is paying for something available at no cost.

Caching as a cost lever

- Every cache hit is a request the backing service never sees, and never bills for.
- **DAX** removes DynamoDB read cost as well as read latency.
- **CloudFront** removes origin requests and reduces egress charges.
- **API Gateway caching** removes backend invocations entirely for cached responses.
- A cache has its own standing cost, so it pays off only above a certain hit rate.

> ON THE EXAM
>
> The break-even is the honest part: a cache that costs 100 dollars a month must remove more than 100 dollars of backend calls, or it is a net loss.

The cost levers, in order of usual impact

LEVER	TYPICAL EFFECT
Set log retention on every group	Often the single largest saving, and free to do
Reduce log verbosity in production	Removes ingestion cost, not just storage
Tune Lambda memory	Frequently 30 to 40% off duration cost
Switch to arm64	20% off duration, one configuration change
Replace NAT gateway with VPC endpoints	Removes a fixed monthly charge entirely
Right-size or commit on steady compute	Savings Plans, reserved capacity, provisioned throughput
Cache what is read repeatedly	Removes requests, above a break-even hit rate

ON THE EXAM

Worth screenshotting. The first two are configuration changes with no engineering effort, which is why they belong at the top.

Right-sizing and commitments

- **Compute Optimizer** recommends instance and Lambda memory sizes from actual usage.
- **Savings Plans** discount compute for a one or three year commitment, including Lambda and Fargate.
- **Spot** for interruption-tolerant work, at a large discount.
- **Reserved capacity** exists for DynamoDB, and reserved nodes for ElastiCache.
- Commitments reward predictability. Committing to capacity you stop using is pure loss.

> ON THE EXAM
>
> Right-size before you commit. A Savings Plan on an oversized fleet locks in the wrong number for a year.

Seeing the cost

- **Cost Explorer** breaks spend down by service, tag, account and time.
- **AWS Budgets** alerts on forecast or actual spend crossing a threshold.
- **Cost Anomaly Detection** flags unusual spend automatically, without a threshold.
- **Trusted Advisor** surfaces idle and underused resources across the account.
- Cost data lags by hours, so none of these catches a runaway within minutes.

> ON THE EXAM
>
> That lag is why a CloudWatch alarm on the underlying metric beats a budget alert for catching a runaway loop. Budgets tell you afterwards.

Attributing the cost

- **Cost allocation tags** must be activated in the billing console before they appear in reports.
- **Activation is not retroactive.** Tags only apply to costs incurred afterwards.
- Tag by team, environment and application to attribute shared-account spend.
- **Tag policies** in Organizations enforce consistent keys and values.
- An IAM condition on **aws:RequestTag** can require tags at creation time.

> ON THE EXAM
>
> The non-retroactive point matters: activating tags today does not explain last month's bill. Tagging has to be in place before the spend happens.

When performance and cost conflict

- **Provisioned concurrency** buys latency with money spent on idle capacity.
- **DAX and ElastiCache** buy latency with a standing cluster cost.
- **Multi-Region** buys resilience and latency with duplicated everything.
- **High-resolution metrics and 100% tracing** buy visibility with ingestion cost.
- In each case the question is whether the latency or visibility is worth the standing charge.

> ON THE EXAM
>
> Not everything aligns. Where the two genuinely conflict, the exam expects you to notice which one the scenario prioritises rather than assuming cheapest wins.

What the exam actually asks

- **Most cost-effective** means the cheapest option that still meets every stated requirement.
- It does **not** mean cheapest overall. An option that drops a requirement is wrong.
- **Least operational overhead** means the most managed option, even if it costs more.
- **Minimal code change** rules out rewrites, however elegant.
- Two options often both work. The qualifier in the question decides between them.

> ON THE EXAM
>
> These phrases are not decoration. They are the tiebreaker, and reading past them is how you pick a defensible answer that is not the intended one.

Ask first: is the money going to compute, to logs, to networking, or to requests?

> An image processing Lambda function is configured with 128 MB of memory and takes about 8 seconds per image. The team wants it faster, and has been told to keep costs down, so they have avoided raising the memory.
>
> The function is CPU-bound, doing image resizing in memory.

What should the team do?

A Keep 128 MB and split the work across several functions invoked in parallel

B Run Lambda Power Tuning, and expect a higher memory setting to be both faster and cheaper

C Enable provisioned concurrency so the function is always warm

D Move the workload to a small EC2 instance running continuously

◆ PAUSE AND DECIDE

What should the team do?

A Split across parallel functions

Each function still gets a fraction of a vCPU, so each piece is still slow. It adds orchestration to avoid a configuration change.

B Power Tuning, expect higher memory

CPU scales with memory, so a CPU-bound function at 128 MB is severely constrained. Finishing several times faster usually costs less in GB-seconds.

C Provisioned concurrency

Removes cold start latency. The 8 seconds is execution time, not initialisation, so this changes nothing and adds cost.

D Move to EC2

Trades per-invocation billing for a continuously running instance. For intermittent image processing that is almost certainly dearer.

THE SIGNAL

The premise is the error: avoiding memory to save money. For a CPU-bound function the opposite is usually true, and Power Tuning is how you prove it.

> A set of Lambda functions is attached to a VPC to reach a private RDS database. They also read and write to Amazon S3 and DynamoDB. The monthly bill shows a substantial NAT gateway charge, with high data processing costs.
>
> The functions do not call anything outside AWS.

How should the networking cost be reduced?

A Remove the functions from the VPC entirely

B Move the NAT gateway to a single Availability Zone

C Replace the NAT gateway with a NAT instance on a small EC2 host

D Add gateway VPC endpoints for S3 and DynamoDB, and remove the NAT gateway

◆ PAUSE AND DECIDE

How should the networking cost be reduced?

A Leave the VPC

The functions need a private RDS database, which requires VPC attachment. This removes the cost by breaking the requirement.

B Single-AZ NAT gateway

Halves or thirds the fixed charge and keeps all the per-GB processing. It also creates an availability single point of failure.

C NAT instance

Cheaper per hour and now something you patch, scale and monitor. It also still processes every byte.

D Gateway endpoints, drop the NAT

Gateway endpoints for S3 and DynamoDB are free with no per-GB charge, and with no external calls the NAT gateway is unnecessary entirely.

THE SIGNAL

'Do not call anything outside AWS' is the clause that allows removing the NAT gateway altogether, rather than merely making it cheaper.

> A company's AWS bill shows CloudWatch costing more than all their Lambda compute combined. Investigation finds debug-level logging enabled in production across 300 functions, and no log group has a retention period set.
>
> The team wants the cost down without losing the ability to debug production issues when they arise.

What should they do?

A. Set retention on every log group, and set the Lambda application log level to INFO so debug lines are never ingested

B. Delete the existing log groups and let them be recreated

C. Move all log groups to the Infrequent Access class

D. Remove the debug logging statements from the code and redeploy

◆ PAUSE AND DECIDE

What should they do?

A Retention plus application log level

Retention removes accumulated storage; the log level stops debug lines being ingested at all. Both are configuration, and the level can be raised again when debugging.

B Delete the log groups

Frees storage once and changes nothing about ingestion, so the cost returns. It also destroys the history.

C Infrequent Access

Reduces the ingestion rate and loses metric filters, subscription filters and alarms. It also cannot be changed back.

D Remove the statements

Requires redeploying 300 functions, and permanently removes the debug logging the team explicitly wants available.

THE SIGNAL

Option D is the near miss. It fixes the cost and destroys the capability the scenario says must be retained; the log level does both without a deployment.

> A DynamoDB table serving a retail catalogue runs in provisioned mode with capacity set for the Black Friday peak. For most of the year utilisation sits below 10%, and the team pays for the peak capacity every month.
>
> Traffic is highly predictable day to day, with two known annual spikes.

What is the best change?

A. Switch permanently to on-demand mode

B. Add a DAX cluster to absorb the peak reads

C. Enable provisioned auto scaling for the daily pattern, and buy reserved capacity for the steady baseline

D. Reduce provisioned capacity to the average and accept throttling at the peak

◆ PAUSE AND DECIDE

What is the best change?

A On-demand permanently

The strongest distractor. On-demand suits unpredictable traffic, and this traffic is described as highly predictable, where provisioned is substantially cheaper.

B Add DAX

Reduces read capacity needs and adds a standing cluster cost. It does nothing about paying for peak write capacity all year.

C Auto scaling plus reserved capacity

Auto scaling follows the predictable daily pattern, and reserved capacity discounts the baseline that is always there. It matches both facts in the scenario.

D Average capacity, accept throttling

Saves money by failing customers at exactly the moment revenue is highest. Throttling on Black Friday is not an acceptable trade.

THE SIGNAL

'Highly predictable' is the deciding phrase. On-demand is right for unpredictable traffic; predictability is precisely what provisioned mode and commitments reward.

What to carry forward

- **In serverless, faster is usually cheaper.** More memory can reduce the bill.
- **Lambda compute is often a minority of a serverless bill.** Check logs and networking first.
- **Gateway endpoints for S3 and DynamoDB are free.** A NAT gateway is not.
- arm64 is 20% off duration for one configuration change and no code change.
- On-demand for unpredictable, provisioned and committed for predictable. Predictability decides.

> ON THE EXAM
>
> That closes the teaching content. Next: confusion pairs, how DVA-C02 questions are written, and the revision gap.